Web Programming

Node - authentication and authorization

Lecture 16

Mahhek Khan

What is authentication?

- Verifying *who* the user is.
- Ensures the user provides valid credentials (e.g., email & password).
- Example: Logging into a website.

What is Authorization?

- Verifying *what* the user is allowed to do.
- Occurs **after** authentication.
- Example: Super admin vs Admin vs regular user access.

Why Authentication and Authorization

Prevents unauthorized access.

Protects sensitive operations (e.g., modifying user data).

Improves app security and user experience

Login → Authentication

Access dashboard → Authorization

2 Approaches

Session based

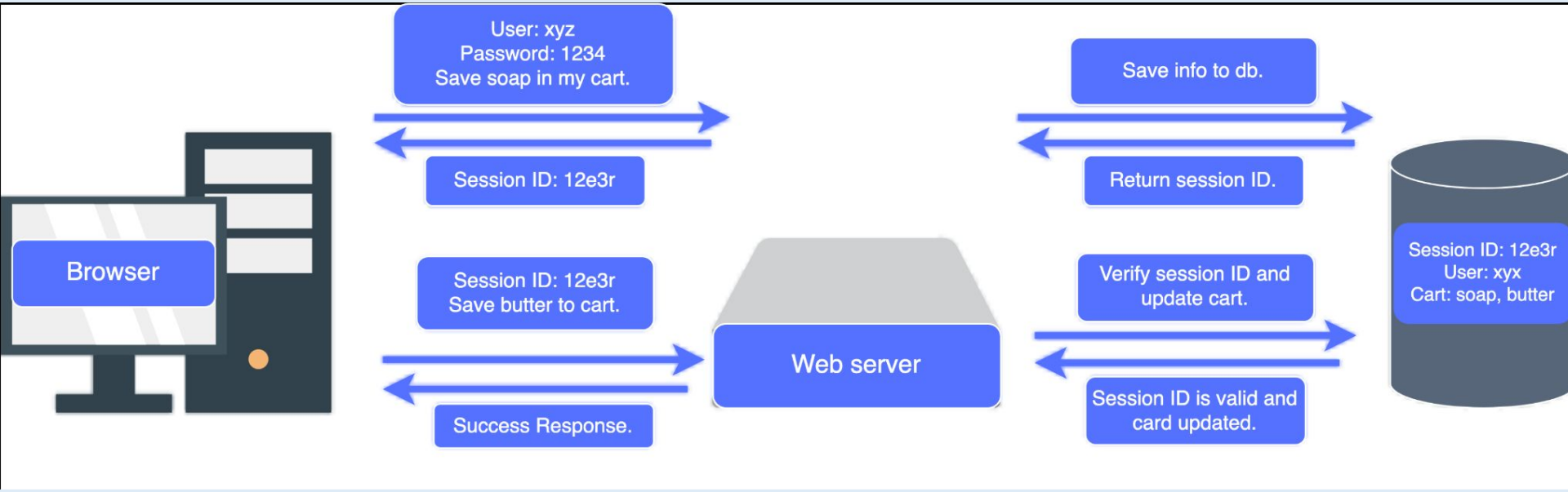
JWT Token

Authentication using session

What is it?

Session-based authentication uses a server-side stored session to track the logged-in user.

1. **User logs in** by submitting their credentials.
2. **Server verifies credentials** and creates a unique session (usually a random ID).
3. **Session data** (user ID, role, etc.) is stored on the server (memory, database, or cache).
4. Server sends a **session ID** to the client via a cookie.
5. On every request, the client **sends back the cookie**, and the server uses the session ID to retrieve the user's data.



npm install dotenv //It installs a package that allows your app to read variables from a `.env` file.

npm install cors // allows your backend to accept requests from a different origin

On node side

resave is just an optimization. It prevents saving the session again if nothing has changed. means only update when we add something to session

saveUninitialized: Don’t create session until we actually store something, means only created after login not on home page.

secure: false //for dev only
httpOnly: This makes cookies safer because frontend JS cannot read them.

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const session = require("express-session");

const app = express();

// Middleware
app.use(express.json());

// CORS
app.use(
  cors({
    origin: "http://localhost:3000",
    credentials: true,
  })
);

// Session
app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      secure: false,
      httpOnly: true,
    },
  })
);
```

On react side

```
// LOGIN REQUEST
fetch("http://localhost:5000/login", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    username: "xyz",
    password: "1234"
  }),
  credentials: "include"
});


// ADD ITEM REQUEST
fetch("http://localhost:5000/add", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    item: "butter"
  }),
  credentials: "include"
});
```

Login → Create Session

```
app.post("/login", (req, res) => {
  const { username, password } = req.body;

  if (username === "xyz" && password === "1234") {
    req.session.user = username;
    req.session.cart = ["soap"];

    res.json({ message: "Logged in" });
  } else {
    res.status(401).json({ message: "Invalid credentials" });
  }
});
```

When we set `req.session`, Express automatically creates a session and sends a cookie to the browser.

The browser stores this cookie, and on every next request, it automatically sends it back.

So React does not manually send the session — the browser handles itas long as we use `credentials: 'include'` in the frontend.

Use Session (Protected Route)

```
app.get("/cart", (req, res) => {
  if (!req.session.user) {
    return res.status(401).json({ message: "Not logged in" });
  }

  res.json({
    user: req.session.user,
    cart: req.session.cart,
  });
});
```

Now we don’t ask for username again, we get it from the session.

Update Session

```
app.post("/add", (req, res) => {
  const { item } = req.body;

  if (!req.session.cart) {
    req.session.cart = [];
  }

  req.session.cart.push(item);

  res.json({
    message: "Item added",
    cart: req.session.cart,
  });
});
```

req.session.cart = req.session.cart.filter(item => item !== "apple");

Authorization using session

req.session.user = "xyz";

req.session.role = "admin";

```
app.get("/admin", (req, res) => {
  if (!req.session.user) {
    return res.status(401).json({ message: "Not logged in" });
  }

  if (req.session.role !== "admin") {
    return res.status(403).json({ message: "Access denied" });
  }

  res.json({ message: "Welcome Admin" });
});
```

JWT-Based Authentication

JSON Web Token.

Server sends a signed **JSON Web Token** after login.

Client stores JWT (commonly in localStorage or cookies).

JWT is sent in the `Authorization` header on requests.

```
npm install jsonwebtoken
```

- User logs in
- Server creates JWT
- Client stores token
- Token sent with every request

Session vs jwt token

Session	JWT
Stored on server	Stored on client
Uses cookies	Uses token (header)
Server remembers user	Token contains data

Session → server stores data

JWT → client stores token

in react

```
const login = async () => {
  const res = await fetch("http://localhost:5000/auth/login", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      username: "xyz",
      password: "1234"
    })
  });

  const data = await res.json();

  localStorage.setItem("token", data.token);
};
```

login.js

`res.json({ token })` sends the token to the client (React/browser)

```
const express = require("express");
const jwt = require("jsonwebtoken");
require("dotenv").config(); // 👈 load env variables

const app = express();
app.use(express.json());

const SECRET = process.env.SECRET; // 👈 from .env

app.post("/auth/login", (req, res) => {
  const { username, password } = req.body;

  if (username === "xyz" && password === "1234") {
    const token = jwt.sign({ user: username }, SECRET, {
      expiresIn: "1h",
    });

    return res.json({ token });
  }

  res.status(401).json({ message: "Invalid credentials" });
});

app.listen(5000, () => {
  console.log("Server running on http://localhost:5000");
});
```

response goes to the client side as

```
{
  "token": "eyJhbGciOiJIUzI1NiIs..."
}
```

in react

```
const token = localStorage.getItem("token");

fetch("http://localhost:3000/user/profile", {
  method: "GET",
  headers: {
    Authorization: token,
  },
});
```

auth.js

```
const jwt = require("jsonwebtoken");
require("dotenv").config();

const SECRET = process.env.SECRET;

const auth = (req, res, next) => {
  const token = req.headers.authorization;

  if (!token) {
    return res.status(401).json({ message: "No token" });
  }

  try {
    const decoded = jwt.verify(token, SECRET);
    req.user = decoded;
    next();
  } catch {
    res.status(401).json({ message: "Invalid token" });
  }
};

module.exports = auth;
```

userRoutes.js

```
const express = require("express");
const router = express.Router();
const auth = require("./auth");

router.get("/profile", auth, (req, res) => {
  res.json({ message: `Welcome ${req.user.user}` });
});
```

JWT based authorization

```
const token = jwt.sign(
  { user: username, role: "admin" },
  SECRET,
  { expiresIn: "1h" }
);
```

```
router.get("/admin", auth, (req, res) => {
  if (req.user.role !== "admin") {
    return res.status(403).json({ message: "Access denied" });
  }

  res.json({ message: "Welcome Admin" });
});
```